Machine Learning for Sound and Music

Lab 2: Convolutional Neural Networks for Music Genre Classification

Lidia Wiśniewska

November 2026

This report summarizes the work presented in the Jupyter notebook.

1 Introduction

Convolutional Neural Networks (CNNs) have become increasingly popular for audio classification tasks, driven by their remarkable success in computer vision. Originally designed to exploit the spatial hierarchies of visual data, CNN architectures have been effectively adapted to analyse time–frequency representations of audio signals, such as spectrograms or Mel-spectrograms. This adaptation leverages the CNNs' ability to learn local and hierarchical features, enabling them to capture important temporal and spectral patterns in sound. Many techniques and architectures that proved successful in vision, such as VGG, ResNet, and Inception, have been transferred to audio analysis, where they demonstrate strong performance in tasks like speech recognition, music genre classification, and environmental sound identification. This cross-domain success illustrates how insights from visual deep learning research have profoundly shaped and accelerated progress in computational audio understanding.

2 Task 1

2.1 Data Analysis

The dataset contains Mel-spectrogram images for ten GTZAN genres: blues, classical, country, disco, hip-hop, jazz, metal, pop, reggae, and rock. Fifteen files identified as duplicates or otherwise excluded in the Lab 1 workflow were removed, leaving 984 samples. The images were cropped to remove the surrounding plot margins. The resulting grayscale images have shape $1 \times 218 \times 336$.

Each image represents one song. The split was stratified by genre to preserve the class distribution:

- 80% of the songs were reserved for training and validation, with the remaining 20% (approximately 197 songs) held out for testing;

- the 80% portion was divided again into training and validation data.

The training and validation sets were used for model selection and hyperparameter tuning. The selected model was then retrained on the combined training and validation data and evaluated on the test set. Lab 1 models were evaluated on the same test songs.

2.2 Baseline Architecture

The baseline is a compact three-block CNN. Each block contains a 3×3 convolution, batch normalization, ReLU activation, and 2×2 max pooling. The blocks use 16, 32, and 64 filters, respectively. Adaptive average pooling reduces the final feature maps to one value per channel. The classifier then applies dropout, a 64-unit hidden layer, ReLU, and a ten-class output layer. The model has 28,218 trainable parameters.

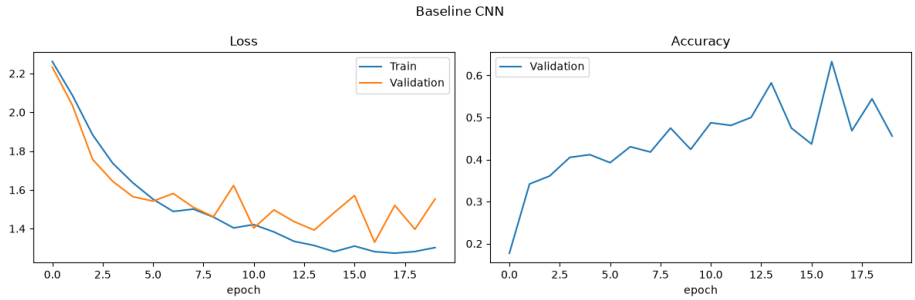


Figure 1: Learning curves of the baseline model

The baseline was trained with Adam, an initial learning rate of 10^{-3}, a batch size of 32, and a maximum of 20 epochs. The checkpoint with the lowest validation loss was retained. The learning curves are seen on Figure 1.

2.3 Experiments and Hyperparameter Tuning

Two small architecture changes were evaluated against the baseline:

1. adding a fourth convolutional block, with channel widths $16, 32, 64, 64$;
2. reversing the widths of the three blocks to $64, 32, 16$.

These variants did not provide a more reliable improvement than the baseline in the validation experiments. Additionally, they produced more unstable training history, one example showed on the learning curves below (Figure 2). Therefore, the baseline architecture was retained for the final scratch CNN.

Hyperparameters for the baseline were then explored using Optuna's Tree-structured Parzen Estimator (TPE) sampler. The same tuning set was performed both for Adam and SGD with momentum=0.9, however Adam yielded

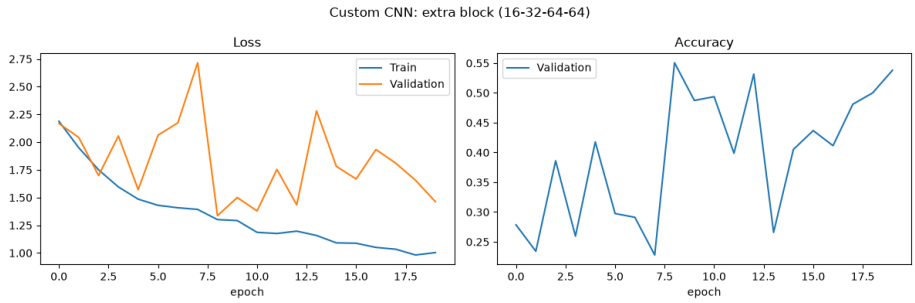


Figure 2: Learning curves of the extended baseline model

better results and was chosen as the optimizer for the rest of tuning. The search covered a log-scaled learning rate from 10^{-4} to 3×10^{-3}, batch sizes of 16, 32, or 64, weight decay from 10^{-6} to 10^{-3}, and dropout values from 0.1 to 0.5 in increments of 0.1. Each trial ran for up to 40 epochs, with early stopping after eight epochs without validation-loss improvement. The lowest validation loss was used to select the configuration.

The selected configuration was:

- optimizer: Adam;
- learning rate: approximately 1.70×10^{-3};
- batch size: 16;
- weight decay: approximately 8.18×10^{-6};
- dropout: 0.3;
- training duration: 27 epochs.

Retrained on the combined training and validation data, the scratch CNN achieved 67.5% test accuracy, with a 95% Wilson confidence interval of 60.7%–73.7%.

3 Task 2

The second task used an ImageNet-pretrained ResNet-18. Its original classification layer was replaced with a ten-class output layer. The experiments considered both a frozen backbone and full fine-tuning, along with different optimizers, learning rates, batch sizes, and training durations. The selected model used full fine-tuning with Adam, a learning rate of 10^{-4}, a batch size of 16, and five epochs.

The fine-tuned ResNet-18 achieved 79.2% test accuracy, with a 95% Wilson confidence interval of 73.0%–84.3%.

4 Comparison with Lab 1

For fair comparison, the models were re-evaluated on the same train/test data split as the CNNs. The best Lab 1 model was selected using validation results, rather than test-set performance. It was the SVM trained on the 30-second hand-crafted descriptors, denoted `svm_30`. Its test accuracy on the common test split was 75.1%.

The table 1 and Figure 3 summarize the test-set results for the models obtained from Lab1 (on the full 30s files) and Lab2.

Table 1: Test accuracy on the seed-42 split

Model	Test accuracy	95% confidence interval
Fine-tuned ResNet-18	**79.2%**	73.0%–84.3%
`svm_30`	**75.1%**	68.6%–80.6%
`lr_30`	74.1%	67.6%–79.7%
`mlp_30`	73.1%	66.5%–78.8%
`hgb_30`	72.6%	66.0%–78.3%
`knn_30`	71.6%	64.9%–77.4%
`rf_30`	71.6%	64.9%–77.4%
CNN trained from scratch	67.5%	60.7%–73.7%
`dt_30`	50.3%	43.3%–57.2%

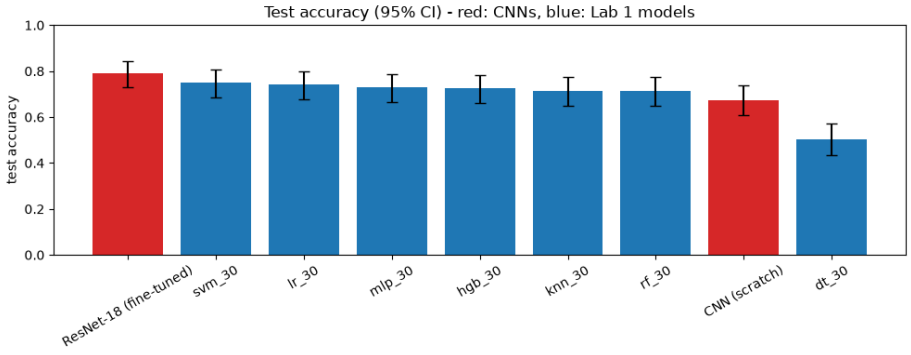


Figure 3: Results of Lab1 and Lab2

5 Research Questions

5.1 Do Learned Features Beat Hand-Crafted Descriptors?

On the seed-42 test split, the scratch CNN achieved 67.5%, compared with 75.1% for `svm_30`. The exact McNemar test did not find a statistically significant

difference between these two models at the 5% level ($p = 0.067$). ResNet-18 achieved 79.2%, but its difference from `svm_30` was also not statistically significant ($p = 0.291$). ResNet-18 did outperform the scratch CNN significantly ($p = 0.001$).

Thus, the pretrained CNN performed best numerically, but this experiment does not show a statistically significant advantage over the best hand-crafted descriptor model. The scratch CNN did not outperform that model.

5.2 Do CNNs and Descriptor Models Make the Same Mistakes?

The models shared more errors than would be expected if their errors were independent. For example, the scratch CNN and `svm_30` both misclassified 27 songs, compared with 15.9 expected under independence. ResNet-18 and `svm_30` both misclassified 23 songs, compared with 10.2 expected.

Among songs that both models misclassified, ResNet-18 and `svm_30` predicted the same incorrect genre 65.2% of the time. This proportion was approximately 48% for the other model pairs. Genre-confusion patterns were positively correlated: the Spearman correlations were 0.59 for scratch CNN versus SVM, 0.68 for ResNet-18 versus SVM, and 0.73 for scratch CNN versus ResNet-18.

These results suggest that the models share some difficult songs and genre confusions, while still making some different errors.

5.3 Are the Two Approaches Complementary?

Late fusion by averaging class probabilities improved the test accuracy of both model pairs:

- scratch CNN + `svm_30`: 80.2%, compared with 67.5% and 75.1% for the individual models;
- ResNet-18 + `svm_30`: 83.2%, compared with 79.2% and 75.1% for the individual models.

For the ResNet–SVM pair, at least one model was correct on 88.3% of test songs, which is an upper bound on the accuracy achievable by a combination that can always choose the correct model.

Early fusion also performed better than either feature set alone. Logistic regression using concatenated CNN embeddings and hand-crafted descriptors achieved 81.7%, compared with 73.6% for descriptors alone and 70.1% for CNN embeddings alone. These results suggest complementarity on this split. However, the fusion results come from a single test split and should be treated as exploratory rather than evidence that fusion will consistently improve performance.

5.4 How Much Does One Train/Test Split Tell Us?

The evaluation was repeated on three stratified splits, with seeds 42, 123, and 8. The ResNet-18 results were comparatively consistent: mean accuracy 78.7%, standard deviation 0.9 percentage points, and range 77.7%–79.2%. The best Lab 1 SVM had mean accuracy 73.6%, standard deviation 1.5 points, and range 72.1%–75.1%. The scratch CNN varied more: mean accuracy 60.6%, standard deviation 6.4 points, and range 54.8%–67.5%.

ResNet-18 ranked first on all three splits, and the SVM outperformed the scratch CNN on all three. The larger variability of the scratch CNN suggests that its result is more sensitive to the particular split or training run. Since only three splits were evaluated, these results provide limited evidence about performance across all possible splits.

6 Conclusions

The compact CNN trained from scratch provided a useful baseline, but its test accuracy (67.5%) was below that of the best Lab 1 descriptor model (75.1%) and the fine-tuned ResNet-18 (79.2%). On the evaluated test split, the statistical comparison did not establish a significant difference between the scratch CNN and the SVM or between ResNet-18 and the SVM. ResNet-18 did significantly outperform the scratch CNN.

The repeated-split results support the same general pattern: ResNet-18 performed strongly and consistently across the three evaluated splits, while the scratch CNN varied more. The fusion experiments also suggest that learned image features and hand-crafted descriptors can provide complementary information, although those results were measured on one split and require further validation.

Overall, the results indicate that pretraining was more effective here than training a small CNN from scratch. They also show that a simple descriptor-based model remains competitive. More repeated splits and independent evaluation of the fusion approaches would be useful before making broader claims.